

地图填色实验报告中的伪代码与结果表

1 暴力回溯伪代码

表 1: 朴素暴力回溯算法伪代码

Algorithm 1 Brute-Force-Backtracking($G = (V, E), k$)

Input: 图 G , 颜色数 k

Output: 合法 k -着色 $color$; 若不存在则返回 failure

```
1  将顶点按固定顺序排列为  $v_1, v_2, \dots, v_n$ 
2  for each  $v \in V$  do  $color[v] \leftarrow \text{UNCOLORED}$  end for
3  function Backtrack( $i$ )
4      if  $i > n$  then return success
5      for  $c = 1$  to  $k$  do
6          if 对任意已着色邻居  $u \in N(v_i)$ , 均有  $color[u] \neq c$  then
7               $color[v_i] \leftarrow c$ 
8              if Backtrack( $i + 1$ ) 成功 then return success
9               $color[v_i] \leftarrow \text{UNCOLORED}$ 
10         end if
11     end for
12     return failure
13 end function
14 return Backtrack(1)
```

表 2: DSATUR 回溯算法伪代码

Algorithm 2 DSATUR-Backtracking($G = (V, E), k$)

Input: 图 G , 颜色数 k **Output:** 合法 k -着色 $color$; 若不存在则返回 failure

```

1  for each  $v \in V$  do
2       $color[v] \leftarrow \text{UNCOLORED}$ 
3       $domain[v] \leftarrow \{1, 2, \dots, k\}$ 
4  end for
5  function Search()
6      if 所有顶点均已着色 then return success
7      选择未着色顶点  $v$ , 使饱和度  $sat(v)$  最大; 若并列, 则取度数  $\deg(v)$  最大者
8      for each  $c \in domain[v]$ , 按前向影响从小到大遍历 do
9          if  $v$  的已着色邻居均未使用颜色  $c$  then
10              $color[v] \leftarrow c$ 
11             从所有未着色邻居  $u$  的  $domain[u]$  中临时删除  $c$ 
12             if 所有未着色邻居的可用颜色集合均非空 and Search() 成功 then
13                 return success
14             end if
15             恢复本轮修改的所有颜色域,  $color[v] \leftarrow \text{UNCOLORED}$ 
16         end if
17     end for
18     return failure
19 end function

```

表 3: TabuCol 局部搜索算法伪代码

Algorithm 3 TabuCol($G = (V, E), k, maxSteps, tabuTenure$)

Input: 图 G , 颜色数 k , 最大迭代步数 $maxSteps$, 禁忌期限 $tabuTenure$

Output: 合法 k -着色 $color$; 若未找到则返回当前最优着色

```

1   $color \leftarrow \text{GREEDYINITIALCOLORING}(G, k)$ 
2   $bestColor \leftarrow color$ 
3   $bestConflict \leftarrow \text{CONFLICTEDGES}(color)$ 
4  初始化禁忌表:  $tabu[v][c] \leftarrow 0$ 
5  for  $step = 1$  to  $maxSteps$  do
6     $currentConflict \leftarrow \text{CONFLICTEDGES}(color)$ 
7    if  $currentConflict = 0$  then return  $color$ 
8    if  $currentConflict < bestConflict$  then
9       $bestConflict \leftarrow currentConflict, bestColor \leftarrow color$ 
10   end if
11    $C \leftarrow \{v \in V \mid v \text{ 至少关联一条冲突边}\}$ 
12   在所有移动  $(v, oldColor \rightarrow newColor)$  中选择评价值最优者
13   其中  $\Delta = \text{移动后冲突边数} - \text{移动前冲突边数}$ 
14   若移动处于禁忌状态, 仅当它优于历史最优解时允许执行
15   执行选中移动:  $color[v] \leftarrow newColor$ 
16   将旧颜色标记为禁忌:  $tabu[v][oldColor] \leftarrow step + tabuTenure + random$ 
17 end for
18 return  $bestColor$ 

```

2 DSATUR 回溯伪代码

3 TabuCol 局部搜索伪代码

4 实验结果表格

4.1 小规模地图正确性测试

表 4: 小规模地图 4 色正确性测试结果

数据	顶点数	边数	颜色数	算法	耗时/s	冲突边
小规模粘贴地图抽象图	9	16	4	DSATUR 回溯	0.000126	0

4.2 朴素暴力回溯与 DSATUR 回溯对比

4.3 三个 450 顶点数据着色结果

4.4 随机平面图规模效率分析

表 5: 朴素暴力回溯与 DSATUR 回溯对比

数据	顶点数	边数	暴力首解/s	暴力赋值	DSATUR/s	DSATUR 赋值	暴力完整空间	外推/s
小规模粘贴地图	9	16	0.000019	19	0.000057	9	2.621×10^5	2.580×10^{-1}
随机平面图 n=12	12	30	0.000022	32	0.000163	12	1.678×10^7	1.143×10^1
随机平面图 n=16	16	42	0.000029	43	0.000105	16	4.295×10^9	2.907×10^3
随机平面图 n=20	20	54	0.000032	51	0.000230	20	1.100×10^{12}	6.985×10^5
随机平面图 n=24	24	66	0.000040	62	0.000172	24	2.815×10^{14}	1.816×10^8
随机平面图 n=28	28	78	0.000040	67	0.000216	28	7.206×10^{16}	4.313×10^{10}
随机平面图 n=32	32	90	0.000047	78	0.000259	32	1.845×10^{19}	1.102×10^{13}

表 6: 三个 DIMACS .col 数据文件的着色结果

数据文件	顶点数	边数	指定颜色数	是否成功	耗时/s	冲突边
le450_5a.col	450	5714	5	成功	1.104681	0
le450_15b.col	450	8169	15	成功	2.501488	0
le450_25a.col	450	8260	25	成功	0.007555	0

表 7: 随机平面图 4 色填色效率

顶点数	边数	颜色数	是否成功	耗时/s	DSATUR 赋值次数
20	54	4	成功	0.000280	20
40	114	4	成功	0.000646	40
80	234	4	成功	0.001980	80
120	354	4	成功	0.003450	120
160	474	4	成功	0.005360	160
220	654	4	成功	0.010082	220
300	894	4	成功	0.020500	300